PRINCIPLES OF APPLIED ENGINEERING · 2026-27

freeCodeCamp JavaScript Track

Mr. R. Gavaldon · Room HS 112
Weekly plan & pacing
verified 2026-08-16

1 What the course actually is

freeCodeCamp replaced the old JavaScript course. The current one is the **JavaScript Certification** (freecodecamp.org/learn/javascript-v9). It is **1,314 steps** across 25 modules, plus **5 required certification projects** and a final exam.

1,314	~66 hrs	5	135
TOTAL STEPS	AT 3 MIN/STEP	CERT PROJECTS	WEEKLY SLOTS NEEDED

Read this before you plan the year

At **one class period per week** you get roughly **35 usable minutes** × 36 weeks = **~21 hours** of coding time. The full certification needs about **66 hours**. **The full certification is not reachable at one day a week** — it would take roughly 135 weekly slots, or about 3.8 school years.

A realistic year-one target: modules 1–5 — Variables and Strings, Booleans and Numbers, Functions, Arrays, and Objects = 268 steps ≈ 25 weekly slots. That leaves about 11 weeks of slack for testing days, assemblies, and short weeks, and it still lands on real ground: students can declare variables, work with strings, write functions, and use arrays and objects.

For reference, going all the way through the *JavaScript Fundamentals Review* (modules 1–7, which adds Loops at 12 weeks) needs about **40 weekly slots** — slightly more than one school year. Loops is the single biggest module in the certification.

2 How freeCodeCamp breaks each module up

Every module is a run of small items, and each item is tagged with a type. This is what makes weekly assignment easy — you assign **whole items**, never a partial one.

TYPE	WHAT IT IS	HOW I WOULD USE IT
Theory	Short readings + questions that teach the concept. No open-ended building.	Whole-class. Project it, work the first one together.
Workshop	A guided build. freeCodeCamp walks them through every step.	The main weekly assignment. Everyone can finish it.
Lab	An independent build. They get the goal, not the steps.	The real assessment. This is where you see who understands.
Review	A recap of everything in the module.	Pair with the Quiz as a single week.
Quiz	A graded check at the end of a module.	Your grade for the six weeks.
Certification Project	One of the 5 required builds (Markdown converter, drum machine, voting system, bank account, weather app).	Give it two weeks. This is a real portfolio piece.

3 Every module, with time

Steps are verified from the live curriculum. Time assumes **3 minutes per step** for a high school student seeing it for the first time (a motivated adult runs about half that). "Weeks" = how many once-a-week class periods at 35 usable minutes.

MODULE	STEPS	EST. TIME	WEEKS
Variables and Strings	101	303 min	9
Booleans and Numbers	54	162 min	5
Functions	38	114 min	4
Arrays	34	102 min	3
Objects	41	123 min	4
Loops	131	393 min	12
JavaScript Fundamentals Review	33	99 min	3
Higher Order Functions and Callbacks	36	108 min	4
DOM Manipulation and Events	87	261 min	8
JavaScript and Accessibility	42	126 min	4
Debugging	8	24 min	1
Basic Regex	46	138 min	4
★ Build a Markdown to HTML Converter	**project**	**~70 min**	**2**
Form Validation	104	312 min	9
Dates	5	15 min	1
Audio and Video Events	54	162 min	5
★ Build a Drum Machine	**project**	**~70 min**	**2**
Maps and Sets	36	108 min	4
★ Build a Voting System	**project**	**~70 min**	**2**
localStorage and CRUD Operations	78	234 min	7
Classes	68	204 min	6
★ Build a Bank Account Management Program	**project**	**~70 min**	**2**
Recursion	119	357 min	11
Data Structures	31	93 min	3
Algorithms	49	147 min	5
Graphs and Trees	50	150 min	5
Dynamic Programming	5	15 min	1
Functional Programming	24	72 min	3
Asynchronous JavaScript	40	120 min	4
★ Build a Weather App	**project**	**~70 min**	**2**

PRINCIPLES OF APPLIED ENGINEERING · 2026-27

freeCodeCamp JavaScript Track

Mr. R. Gavaldon · Room HS 112
Weekly plan & pacing
verified 2026-08-16

4 The weekly format

Same shape every week so students walk in knowing what to do. Fits a 45-minute period.

TIME	WHAT HAPPENS
5 min	**Bell work** — one question about last week's concept, in the usual format-on-top / example-on-bottom layout.
3 min	**Model it** — you project the first item and work step one with them.
32 min	**The block** — students work the assigned freeCodeCamp item(s) to completion.
5 min	**Exit ticket** — one short answer + a screenshot of their completed item posted to Google Classroom.

Turn-in, every week

- Screenshot of the completed item showing the green check.
- The exit-ticket answer typed in the Classroom comment.
- Nothing else. It stays a 30-second grade for you.

Two things to set up once

- **Accounts:** students sign in to freeCodeCamp with their school Google account so progress saves. Do this the same day you finish Fusion account setup.
- **Progress is visible:** have them keep the freeCodeCamp profile public — you can then see completed items without collecting anything.

5 Semester 1 — the first nine blocks (Module 1: Variables and Strings)

These are the real items in freeCodeCamp's order. Each row is **one week** and is fully completable in the period.

WK	ASSIGN	TYPE	BELL WORK	EXIT TICKET
1	**Introduction to JavaScript · Introduction to Strings · Understanding Code Clarity** What JavaScript is, how it works with HTML/CSS, data types, variables, let vs const.	Theory x3	What is a variable, and why does a program need one?	Write one line of JavaScript that stores your name in a variable.
2	**Build a Greeting Bot** First guided build - takes input and prints a greeting.	Workshop	What is the difference between let and const?	Paste the line of your Greeting Bot that prints the message.
3	**Build a JavaScript Trivia Bot · Build a Sentence Maker** Two independent builds - no step-by-step hand-holding.	Lab x2	In a Lab, freeCodeCamp does not give you the steps. What do you do first when you get stuck?	Which lab was harder, and what was the hardest part?

4	**Working with Data Types · Review · Quiz** Wrap up variables and data types, then the module quiz.	Theory + Review + Quiz	Name three JavaScript data types and give an example value for each.	Your quiz score, and one question you got wrong and now understand.
5	**Working with Strings in JavaScript · Build a Teacher Chatbot** String basics, then a guided chatbot build.	Theory + Workshop	What is a string, and how do you join two strings together?	Paste one line from your Teacher Chatbot.
6	**String Character Methods · Search and Slice Methods · Build a String Inspector** Reaching into a string: characters, searching, slicing.	Theory x2 + Workshop	If a string is "ENGINEER", what is at position 0?	Write one line that pulls the first 3 letters out of a word.
7	**String Formatting Methods · Build a String Formatter** Upper case, lower case, trimming, padding.	Theory + Workshop	Why would a program need to change text to all lower case?	Show one line of your String Formatter and say what it does.
8	**String Modification Methods · Build a String Transformer** Replacing and rebuilding strings.	Theory + Workshop	What does it mean to replace part of a string?	Paste your String Transformer output before and after.
9	**JavaScript Strings Review · JavaScript Strings Quiz** Close out Module 1 with the strings review and quiz.	Review + Quiz	Which string method has been the most useful so far, and why?	Your quiz score + the method you still need to practice.

After these nine weeks: **Booleans and Numbers** (5 wks) → **Functions** (4 wks) → **Arrays** (3 wks) → **Objects** (4 wks) → **Loops** (12 wks) → **Fundamentals Review** (3 wks). Same block pattern: Theory + Workshop most weeks, a Lab week, then Review + Quiz to close the module.

6 How this fits with the Fusion project

One coding day a week, all year

- Pick **one fixed day** — the same day every week so it becomes routine ("Code Day").
- Fusion runs the other four days. Neither one waits on the other, so a slow Fusion week never costs you a coding week.
- During a Fusion certification push, coding day still runs — it is the built-in brain break.
- If a week gets eaten by testing or an assembly, you skip that block; nothing downstream breaks because each block is self-contained.

7 TEKS alignment — Principles of Applied Engineering §127.391

STUDENT EXPECTATION	TEXT	HOW THE JAVASCRIPT WORK MEETS IT
§127.391(d)(1)(B)	identify the inputs, processes, and outputs associated with technological systems	Every program is literally input → process → output. Name the three in each build.
§127.391(d)(5)(C)	use problem-solving techniques to develop technological solutions such as product, process, or system	Labs and certification projects are open-ended problems solved in code.
§127.391(d)(2)(F)	use collaborative tools such as desktop or web-based applications to share and develop information	freeCodeCamp is the web-based platform; work is submitted through Google Classroom.
§127.391(d)(2)(A)	use clear and concise written, verbal, and visual communication techniques	Exit tickets require explaining code in plain English.

§127.391(d)(10)(A–I)	identify and define an engineering problem … test and evaluate proposed solutions … document decisions	The five certification projects follow this cycle end to end.

Be aware

§127.391 has **no explicit programming or coding student expectation** — it is a survey course. The alignments above are real and defensible, but if you are ever asked to justify a whole coding strand, the honest answer is that it supports the systems, problem-solving, and communication expectations rather than a coding expectation.

If you ever want a course where coding *is* the standard, Texas added **§127.403 Programming for Engineers (One Credit, Adopted 2025)** in the same subchapter.

Curriculum structure and step counts pulled from freecodecamp.org/learn/javascript-v9 on 2026–08–16. TEKS text from the TEA currently-effective 19 TAC Chapter 127, Subchapter I (Engineering).